Universidad ORT Uruguay

Facultad de Ingeniería

AIOps

Obligatorio

Plan de contención de incidentes operacionales de PharmaGo

Objetivo

Definir cómo se detectan, priorizan, mitigan y analizan los incidentes operacionales de PharmaGo, siguiendo el proceso de incident management de Atlassian.

Santiago Remedi - 281190

Tomás Zabalza - 256363

Pedro Azambuja - 270218

Docente: Juan Ignacio Irabedra

Fecha de entrega: 16 de noviembre de 2026

Objetivo y alcance

Este plan define cómo se detectan, priorizan, mitigan y analizan los incidentes operacionales de PharmaGo, para reducir el impacto en los usuarios y recuperar el servicio lo antes posible. Sigue el proceso de incident management de Atlassian y cubre los servicios que corren en Kubernetes, en el namespace pharmago.

Los servicios críticos son el API Gateway, por donde entra todo el tráfico, el Users Service (login y usuarios), el Pharmacy Service (medicamentos, compras y stock), la base de datos SQL Server y el frontend. La telemetría (Prometheus, Grafana, Elasticsearch y Kibana) no afecta al usuario, pero sin ella el equipo pierde visibilidad.

Niveles de severidad

La severidad mide el impacto en el usuario. La asigna el Incident Commander al declarar el incidente y, ante la duda, se elige la más alta.

Nivel	**Descripción**	**Ejemplo en PharmaGo**
SEV1	Crítico. El servicio está caído para todos los usuarios o hay riesgo de pérdida de datos.	Se cae el gateway o la base de datos y nadie puede usar la aplicación.
SEV2	Mayor. Una función central no anda o falla para parte de los usuarios.	Falla el login o no se pueden registrar compras.
SEV3	Menor. Molestia leve o degradación con la que se puede seguir trabajando.	Un pod con CPU o memoria alta, sin errores para el usuario.

SEV1 y SEV2 se atienden de inmediato con todo el equipo y llevan postmortem. SEV3 lo atiende la persona de guardia.

Roles

Rol	Responsabilidad	Responsable
Incident Commander	Coordina la respuesta, asigna tareas, toma las decisiones y comunica el estado del incidente.	Santiago Remedi
Responders	Investigan la causa y ejecutan las acciones de mitigación.	Pedro Azambuja y Tomás Zabalza
Stakeholders	Reciben información sobre el incidente y no operan.	Usuarios y responsables de PharmaGo

La guardia (on-call) rota cada semana entre los integrantes. Quien está de guardia recibe las alertas y actúa como Incident Commander hasta que delega el rol.

Comunicación

Durante un incidente toda la comunicación pasa por un único canal, la war room, que es una llamada y un chat del equipo. Las reglas son las siguientes.

- Un solo canal, sin conversaciones paralelas.
- Roles claros. El Incident Commander decide y nadie hace un cambio sin avisar.
- Comunicación clara, frecuente y basada en hechos, citando la alerta, el panel o el log.
- Cada decisión queda registrada con su hora.

Proceso

El proceso sigue los pasos de incident management de Atlassian.

Paso	Qué se hace	Herramientas
Detect	Una alerta de Grafana avisa del problema. La persona de guardia confirma en los dashboards que es real y anota la hora.	Grafana, Prometheus
Respond	Se declara el incidente, se asignan la severidad y los roles, se abre el registro y se elige el playbook. Si hace falta, se escala temprano.	Grafana, Kibana, kubectl
Recover	Se aplica la mitigación (reinicio, rollback, escalado o degradación controlada) y se verifica que el servicio volvió, que está estable y que no quedan usuarios afectados.	kubectl, Grafana
Learn	Se hace un postmortem blameless con la línea de tiempo y la causa raíz.	Template de postmortem
Improve	Las acciones correctivas se cargan como tareas con responsable y fecha, para que el incidente no se repita.	GitHub Issues

Playbooks

Cada playbook indica cómo se detecta el incidente, qué hacer y cómo verificar que se resolvió. Los ejecutan los responders, con la aprobación del Incident Commander.

Incidente	Cómo se detecta	Qué hacer	Cómo se verifica
Exceso de requests	Pico de tráfico y errores 429 en Grafana.	El rate limiting del gateway frena el exceso. Si el gateway queda con errores, reiniciarlo con kubectl rollout restart.	El tráfico y los errores vuelven a lo normal.
Sobrecarga de CPU	Alerta de CPU alta en un pod.	Ubicar el pod con kubectl top pod y reiniciar su deployment con kubectl rollout restart.	La CPU vuelve a lo normal y el pod está Ready.
Sobrecarga de memoria	Alerta de memoria alta o reinicios del pod por OOMKilled.	Kubernetes reinicia el contenedor. Si sigue alta, reiniciar el deployment, y si empezó con un despliegue, hacer kubectl rollout undo.	La memoria baja y el pod deja de reiniciarse.
Disco lleno	Alerta de storage anómalo o poco espacio libre en el nodo.	Borrar los archivos que sobran o reiniciar el pod, que arranca con el disco limpio.	El espacio libre vuelve a lo normal.
Corte de red	Errores 502 o 503 en el gateway y tráfico de red en cero en un pod.	Revisar los Services con kubectl get svc,endpoints y reiniciar el pod afectado.	Las requests vuelven a responder bien.
Caída de un componente	Alerta de deployment sin réplicas o errores 5xx por la base de datos.	Levantarlo con kubectl scale --replicas=1 o borrar el pod colgado para que Kubernetes lo recree.	El deployment tiene réplicas disponibles y no hay errores.

Template de registro de incidente

Se completa durante el incidente.

Inicio (fecha y hora)	
Descripción	
Servicios afectados	
Severidad	
Acciones tomadas (con hora)	

Template de postmortem

Se completa después de cada SEV1 y SEV2, sin buscar culpables y con foco en el sistema.

Qué ocurrió	
Impacto	
Línea de tiempo	
Causa raíz	
Acciones correctivas	